Exercícios — Trabalhando com arquivos JSON em Python

Objetivo

Neste exercício, você irá trabalhar com arquivos **JSON** em Python e utilizá-los para armazenar e recuperar informações.

Você irá praticar:

- Estrutura de arquivos JSON;
- Leitura de arquivos;
- Módulos Python;
- Funções;
- Dicionários;
- Tratamento de erros com `try/except`;
- Acesso a dados utilizando chaves;
- Cálculos utilizando dados externos ao programa.

Exercício 1 — Criar o arquivo de dados JSON

1. Crie o arquivo

No VS Code, crie um novo arquivo e salve-o como:

```
insulin.json
```

O arquivo deverá ser salvo dentro da pasta:

```
files
```

2. Adicione os dados

Copie para o arquivo o conteúdo JSON fornecido no exercício.

Observe sua estrutura antes de executar qualquer coisa.

O arquivo possui três partes principais:

```
molecules
weights
molecularWeightInsulinActual
```

`molecules`

Contém as diferentes sequências da insulina.

`weights`

Contém o peso de cada aminoácido.

Por exemplo, existe uma relação entre uma letra e um valor numérico:

```
"A" → 89.09
```

`molecularWeightInsulinActual`

Armazena o peso molecular real da insulina.

> **Dica:** JSON utiliza uma estrutura semelhante aos dicionários do Python, utilizando pares de **chave e valor**.

Exercício 2 — Criar o módulo manipulador de JSON

Objetivo

Agora você criará um módulo responsável por **ler o arquivo JSON** e disponibilizar seus dados para outros arquivos Python.

1. Abra o arquivo `jsonFileHandler.py`

Esse arquivo será responsável por trabalhar com o arquivo JSON.

2. Importe o módulo JSON

No início do arquivo, importe o módulo padrão responsável por trabalhar com JSON.

```
import json
```

3. Crie a função

Crie uma função chamada:

```
readJsonFile
```

Ela deverá receber o nome do arquivo como argumento.

> **Dica:** o nome recebido pela função será utilizado posteriormente pelo `open()`.

4. Crie uma variável para armazenar os dados

Dentro da função, crie uma variável inicialmente vazia.

Essa variável será utilizada para armazenar os dados recuperados do arquivo.

5. Abra e leia o arquivo

Utilize:

```
open()
```

para abrir o arquivo recebido pela função.

Depois, utilize o módulo `json` para transformar o conteúdo do arquivo em uma estrutura que possa ser manipulada pelo Python.

> **Dica:** a função responsável por carregar um arquivo JSON transforma seu conteúdo em estruturas Python, como dicionários e listas.

Utilize também `with` para trabalhar com o arquivo.

6. Adicione tratamento de erro

Utilize um bloco:

```
try:
    ...
except:
    ...
```

Caso o arquivo não possa ser aberto, mostre:

```
Could not read file
```

A função deverá retornar os dados recuperados quando a leitura funcionar.

Caso ocorra um erro na abertura do arquivo, o valor retornado deverá permanecer vazio.

> **Dica:** utilize `IOError` no `except` para tratar problemas relacionados à leitura ou abertura do arquivo.

Exercício 3 — Criar o programa principal

Objetivo

Agora você irá utilizar o módulo criado anteriormente para recuperar os dados do JSON e calcular o peso molecular da insulina.

1. Abra `calc_weight_json.py`

Esse será o arquivo principal do programa.

2. Importe seu módulo

Importe o módulo:

```
jsonFileHandler
```

Dessa forma, seu programa poderá utilizar a função criada anteriormente.

3. Leia o arquivo JSON

Utilize a função `readJsonFile()` para ler:

```
files/insulin.json
```

Armazene o resultado em uma variável chamada `dados`.

> **Dica:** observe que a função está em outro arquivo. Para utilizá-la, você deverá acessar a função por meio do módulo importado.

4. Verifique se os dados foram recuperados

Antes de tentar acessar as informações do JSON, verifique se a variável `dados` não está vazia.

Utilize uma estrutura condicional para isso.

Caso os dados tenham sido carregados corretamente, recupere:

- `bInsulin`;
- `aInsulin`;
- o peso molecular real da insulina.

Acesso aos dados

Lembre-se de que o JSON possui estruturas dentro de estruturas.

Por exemplo:

```
dados
  └── molecules
       └── bInsulin
```

Você precisará acessar as chaves seguindo essa hierarquia.

> **Dica:** como os dados foram convertidos para um dicionário Python, utilize as chaves entre `[]` para acessar os valores.

5. Combine as cadeias da insulina

Crie uma variável `insulin` combinando:

```
bInsulin
```

e:

```
aInsulin
```

Utilize o operador `+` para concatenar as duas strings.

6. Exiba os dados recuperados

Utilize `print()` para mostrar:

```
bInsulin: ...
aInsulin: ...
molecularWeightInsulinActual: ...
```

Para exibir o peso molecular, lembre-se de que ele é um valor numérico.

> **Dica:** se você estiver concatenando esse valor com uma string utilizando `+`, será necessário convertê-lo para texto.

7. Teste o programa

Execute o programa e verifique se os dados foram recuperados corretamente.

O resultado deverá apresentar as sequências de `bInsulin` e `aInsulin` e o peso molecular real.

---

8. Teste um arquivo inexistente

Agora faça um pequeno teste de tratamento de erros.

Altere temporariamente o nome do arquivo para algo que não exista, por exemplo:

```
files/insuline.json
```

Execute novamente.

Você deverá receber uma mensagem indicando que o arquivo não pôde ser lido e, posteriormente, uma mensagem indicando que o programa será encerrado.

Depois do teste, **volte o nome para `files/insulin.json`**.

---

9. Calcule o peso molecular

Agora você utilizará os dados armazenados no JSON para realizar o cálculo.

Dentro da condição em que os dados foram carregados corretamente:

Recupere os pesos

Os pesos dos aminoácidos estão armazenados na chave:

```
weights
```

Armazene esse dicionário em uma variável.

Conte os aminoácidos

Utilize a sequência armazenada em `insulin` para descobrir quantas vezes cada aminoácido aparece.

O código fornecido utiliza:

```
count()
```

para realizar essa contagem.

> **Dica:** observe também o uso de `upper()` para garantir que a sequência seja analisada em letras maiúsculas.

Calcule o peso

Para cada aminoácido:

```
quantidade × peso
```

Depois, some os valores obtidos para chegar ao peso molecular aproximado.

O código fornecido utiliza `sum()` e uma compreensão de dicionário para realizar essa etapa.

10. Exiba o resultado

Mostre no console:

```
The rough molecular weight of insulin: valor
```

O resultado esperado será aproximadamente:

```
6696.42
```

---

11. Calcule o percentual de erro

Por fim, utilize o peso calculado e o peso real armazenado no JSON para calcular o percentual de erro.

Utilize a fórmula:

```
|peso calculado - peso real| / peso real × 100
```

Exiba o resultado no formato:

```
Percent error: valor
```

O resultado deverá ser aproximadamente:

```
15.30%
```

> **Dica:** o código fornecido utiliza diretamente a diferença entre os valores. Observe a fórmula utilizada e compare-a com a definição de percentual de erro apresentada no exercício.

Checklist

Arquivo JSON

- Criei `insulin.json`.
- Salvei o arquivo dentro da pasta `files`.
- Entendi a estrutura `molecules`.
- Entendi a estrutura `weights`.
- Identifiquei o peso molecular real armazenado no JSON.

Módulo `jsonFileHandler`

- ImporteI o módulo `json`.
- Criei `readJsonFile()`.
- Utilizei `open()` para abrir o arquivo.
- Converti o conteúdo JSON para uma estrutura Python.
- Utilizei `try/except`.

- Tratei um arquivo que não pode ser encontrado.
- Retornei os dados para o programa principal.

Programa principal

- Importei `jsonFileHandler`.
- Recuperei os dados do JSON.
- Acessei os dados utilizando suas chaves.
- Combinei `bInsulin` e `aInsulin`.
- Recuperei os pesos dos aminoácidos.
- Contei os aminoácidos da sequência.
- Calculei o peso molecular aproximado.
- Calculei o percentual de erro.
- Executei o programa e conferi os resultados.